

BO CAU HOI VAN DAP BAO VE DO AN

ASP.NET Core Web API — Ma SV: 0214268

Phan tich toan bo source code repo 68CS20214268

50+ cau hoi • 4 muc do kho • Kich ban van dap thuc te • Cau bay

Tai lieu duoc xay dung dua tren phan tich truc tiep source code — Bam sat 100% vao code thuc te

MUC LUC

P h a n	Noi dung	So cau
1	Kien truc & Tong quan du an	8
2	Entities & Database Design	6
3	Controllers & HTTP	5
4	Services & Interfaces — Dependency Injection	6
5	DTOs & Validation	5
6	DbContext, Migrations & EF Core	6
7	Exceptions, Utils, Constants	4
8	Cau HOI BAY — Giang vien dung de kiem tra hieu biet that	6
9	KICH BAN VAN DAP THUC TE (30-cau lien tiep)	30

PHẦN 1. KIẾN TRÚC & TỔNG QUAN DỰ ÁN

Câu 1

Co ban

CAU HỎI

Dự án này sử dụng công nghệ gì? Hãy nêu tên Framework, ngôn ngữ, ORM và phiên bản.

File: Program.cs / 68CS20214268.csproj

MỤC ĐÍCH HỎI

Kiểm tra sinh viên có biết công nghệ dùng trong dự án không.

DÁP AN MẪU

Dự án dùng C# trên .NET 10, framework ASP.NET Core Web API, ORM là Entity Framework Core với provider Microsoft.EntityFrameworkCore.SqlServer, database SQL Server. Tài liệu API được tạo tự động bằng Swagger (Swashbuckle).

```
// file: 68CS20214268.csproj
<TargetFramework>net10.0</TargetFramework>
// file: Program.cs
builder.Services.AddDbContext<ApplicationDbContext0214268De2>(
    options => options.UseSqlServer(...));
builder.Services.AddSwaggerGen();
```

SAI LAM THƯỜNG GẶP

Nói sai phiên bản .NET (nói .NET 6/7/8 thay vì 10). Không biết Swashbuckle là gì.

CAU HOI

Mo ta kien truc layered (phan tang) cua du an. Moi tang co nhiem vu gi?

MUC DICH HOI

Kiem tra sinh vien hieu kien truc tong the, biet tai sao phan tang.

DAP AN MAU

3 tang chinh:

- Presentation Layer → Controllers/ — nhan HTTP request, tra HTTP response
- Business Logic Layer → Services/Interfaces/ + Services/Implements/ — xu ly nghiep vu, validate, mapping
- Data Access Layer → DbContexts/ + Entities/ + Migrations/ — giao tiep SQL Server qua EF Core

Ho tro: Dtos/ (truyen du lieu), Exceptions/ (xu ly loi), Utils/ (tien ich), Constants/ (hang so)

```

Controllers/
└─ DoanhNghiepController0214268De2.cs
Services/
├─ Interfaces/ IDoanhNghiepService0214268De2.cs
└─ Implements/ DoanhNghiepService0214268De2.cs
DbContexts/
└─ ApplicationDbContext0214268De2.cs
Entities/ / Dtos/ / Migrations/ ...
    
```

SAI LAM THUONG GAP

Mo ta chung chung khong biet ten cu the. Lan lon tang "Service" voi tang "Controller".

CAU HOI

Dependency Injection (DI) hoạt động ra sao trong dự án này? Chỉ ra dòng code cụ thể.

File: Program.cs & DoanhNghiepController0214268De2.cs

MUC DICH HOI

Kiểm tra hiểu biết về IoC container, AddScoped và constructor injection.

DAP AN MAU

DI là cơ chế để các đối tượng nhận các phụ thuộc từ bên ngoài thay vì tự tạo ra.

- Đăng ký (Registration) trong Program.cs:

```
// Program.cs dòng 15-16
builder.Services.AddScoped<IDoanhNghiepService0214268De2,
    DoanhNghiepService0214268De2>();
builder.Services.AddDbContext<ApplicationDbContext0214268De2>(
    options => options.UseSqlServer(...));
```

- Tiêu thụ (Consumption) qua constructor:

```
// DoanhNghiepController0214268De2.cs
private readonly IDoanhNghiepService0214268De2 _doanhNghiepService;
public DoanhNghiepController0214268De2(
    IDoanhNghiepService0214268De2 doanhNghiepService)
{
    _doanhNghiepService = doanhNghiepService; // DI inject
}
```

AddScoped: mỗi HTTP request tạo 1 instance mới, dùng chung trong 1 request.

SAI LAM THUONG GAP

Nhầm AddScoped với AddSingleton/AddTransient. Không biết tại sao cần DI.

CAU HOI

Program.cs thực hiện những cấu hình gì khi khởi động ứng dụng?

File: Program.cs

MUC DICH HOI

Kiểm tra sinh viên đọc và hiểu file Program.cs.

DAP AN MAU

Program.cs thực hiện theo thứ tự:

1. AddControllers() — đăng ký MVC Controllers
2. AddEndpointsApiExplorer() + AddSwaggerGen() — Swagger UI
3. AddDbContext với SQL Server + connection string từ appsettings
4. AddScoped cho IDoanhNghiepService
5. Middleware pipeline: UseSwagger, UseSwaggerUI (chỉ Dev), UseAuthorization
6. MapGet("/") redirect sang "/swagger"
7. MapControllers() + Run()

```
builder.Services.AddControllers();
builder.Services.AddSwaggerGen();
builder.Services.AddDbContext<...>(opt => opt.UseSqlServer(...));
builder.Services.AddScoped<IDoanhNghiepService0214268De2,
    DoanhNghiepService0214268De2>();
app.UseSwagger(); app.UseSwaggerUI();
app.MapGet("/", ctx => { ctx.Response.Redirect("/swagger"); });
app.MapControllers();
app.Run();
```

SAI LAM THUONG GAP

Bỏ qua middleware pipeline. Không biết thứ tự quan trọng.

CAU HOI

appsettings.json chua gi? Connection string trong do la gi?

File: appsettings.json

MUC DICH HOI

Kiem tra sinh vien biet cau hinh ung dung, bao mat connection string.

DAP AN MAU

appsettings.json chua:

```
{
  "Logging": { "LogLevel": { "Default": "Information" } },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost;
    Database=68CS20214268Db;
    Trusted_Connection=True;
    TrustServerCertificate=True"
  }
}
```

Connection string gom: Server (localhost), Database (68CS20214268Db), Trusted_Connection (Windows Auth), TrustServerCertificate (tranh loi SSL local).

appsettings.Development.json chi override Logging khi ASPNETCORE_ENVIRONMENT=Development.

SAI LAM THUONG GAP

Doc nhảm gia tri. Khong biet TrustServerCertificate de lam gi.

CAU HOI

Tai sao chay "dotnet run" lai tu mo Swagger thay vi trang trong?

File: Program.cs

MUC DICH HOI

Kiem tra hieu biet ve middleware MapGet va redirect.

DAP AN MAU

Trong Program.cs co dong redirect:

```
app.MapGet("/", async context =>
{
    context.Response.Redirect("/swagger");
    await Task.CompletedTask;
});
```

Khi client goi GET "/" , middleware bắt và trả HTTP 302 Redirect tới "/swagger". Swagger UI sau đó được render từ UseSwaggerUI() đã đăng ký trước đó.

SAI LAM THUONG GAP

Nghi là Swagger tự mở mặc định. Không biết đây là redirect thủ công.

CAU HOI**Luong request di tu dau den cuoi nhu the nao khi client goi POST /api/DoanhNghiep?***File: DoanhNghiepController0214268De2.cs***MUC DICH HOI**

Kiem tra sinh vien hieu toan bo pipeline xu ly request.

DAP AN MAU

Luong day du:

1. Client gui: POST /api/DoanhNghiep
Body: {"tenDoanhNghiep":"Cty A","maSoThue":"123","diaChi":"HN"}
2. ASP.NET Core routing:
Route [Route("api/[controller]")] + [HttpPost]
→ DoanhNghiepController.Create()
3. [ApiController] tu dong:
 - Bind JSON → CreateDoanhNghiepDto
 - Setter Dto tu Trim ky tu trang
 - Validate DataAnnotations ([Required],[MaxLength])
 - Neu fail → tra 400 Bad Request tu dong
4. Controller goi:
var result = await _doanhNghiepService.CreateAsync(input);
5. Service:
 - Kiem tra trung TenDoanhNghiep (AnyAsync)
 - Kiem tra trung MaSoThue (AnyAsync)
 - Tao entity moi, _context.Add(entity)
 - SaveChangesAsync() → SQL INSERT
 - Map entity → DTO
6. Controller: return Ok(result)
7. Client nhan: HTTP 200 + JSON DoanhNghiepDto

SAI LAM THUONG GAP

Bo qua buoc validate cua [ApiController]. Khong biet setter Dto tu Trim.

CAU HỎI

AddScoped, AddSingleton, AddTransient khác nhau thế nào? Ưu nhược điểm của chúng là gì?

File: Program.cs

MỤC ĐÍCH HỎI

Kiểm tra kiến thức sâu về DI lifetime và tại sao chọn AddScoped.

ĐÁP ÁN MẪU

- AddTransient: tạo instance MỚI mỗi lần được inject (ke cả trong cùng request)
- AddScoped: tạo 1 instance cho MỖI HTTP request, dùng chung trong request đó
- AddSingleton: tạo 1 instance DUY NHẤT cho cả ứng dụng, dùng chung mọi request

Ưu nhược điểm AddScoped cho cả DbContext và DoanhNghiepService. Lý do:

DbContext giữ Unit of Work (theo dõi thay đổi) — phải Scoped để chắc chắn mỗi request có DbContext riêng, tránh race condition giữa các request đồng thời.

Service phụ thuộc DbContext (Scoped) nên Service cũng phải Scoped (không thể inject Scoped vào Singleton).

SAI LAM THƯỜNG GẶP

Nhầm Scoped với Singleton. Không giải thích được tại sao DbContext cần Scoped.

PHẦN 2. ENTITIES & DATABASE DESIGN

Câu 9

Co ban

CAU HOI

BaseEntity0214268De2 chứa những trường gì? Tại sao cần lớp cơ sở này?

File: Entities/BaseEntity0214268De2.cs

MỤC DICH HOI

Kiểm tra hiểu biết về kế thừa, code reuse và soft delete pattern.

DAP AN MAU

BaseEntity0214268De2 (abstract class) chứa:

```
public int Id { get; set; }           // PK auto-increment
public DateTime CreatedAt { get; set; } // = DateTime.UtcNow
public DateTime? UpdatedAt { get; set; } // null nếu chưa update
public bool IsDeleted { get; set; } = false; // soft-delete
```

Lý do cần: tránh viết lặp 4 trường này ở mọi entity (DRY principle). Ngoài ra DbContext dùng reflection để đặt global query filter tự động cho tất cả entity kế thừa BaseEntity.

SAI LAM THUONG GAP

Không biết IsDeleted để làm gì. Không giải thích được global query filter.

CAU HOI

Soft delete là gì? Nó được implement như thế nào trong dự án này?

File: Entities/BaseEntity0214268De2.cs & DbContexts/ApplicationDbContext0214268De2.cs

MỤC DICH HOI

Kiểm tra hiểu biết về soft delete pattern và global query filter.

DAP AN MAU

Soft delete: thay vì xóa vật lý (DELETE SQL), chỉ đặt có IsDeleted = true. Dữ liệu vẫn tồn tại trong DB, có thể khôi phục sau.

Implement trong dự án:

1. BaseEntity có trường IsDeleted = false mặc định
2. DbContext đặt global query filter cho MỌI entity kế thừa BaseEntity:

```
// ApplicationDbContext0214268De2.cs
foreach (var entityType in modelBuilder.Model.GetEntityTypes())
{
    if (typeof(BaseEntity0214268De2).IsAssignableFrom(entityType.ClrType))
    {
        // Expression: e => e.IsDeleted == false
        modelBuilder.Entity(entityType.ClrType).HasQueryFilter(filter);
    }
}
```

3. DeleteAsync trong Service đặt IsDeleted = true, KHÔNG dùng Remove():

```
// DoanhNghiepService0214268De2.cs
entity.IsDeleted = true;
_context.DoanhNghieps.Update(entity);
await _context.SaveChangesAsync();
```

SAI LAM THUONG GAP

Nghi Delete là xóa thật sự. Không biết global query filter là gì, hoạt động ra sao.

CAU HOI

Quan he giữa DoanhNghiep và SanPham là gì? Tại sao cần bảng trung gian DoanhNghiepSanPham?

File: Entities/DoanhNghiepSanPham0214268De2.cs & DbContexts/

MUC DICH HOI

Kiểm tra hiểu biết về quan hệ Many-to-Many và khi nào cần entity trung gian.

DAP AN MAU

Quan hệ Many-to-Many: một DoanhNghiep nhập nhiều SanPham, một SanPham có thể được nhiều DoanhNghiep nhập.

Cần entity trung gian DoanhNghiepSanPham0214268De2 vì ngoài việc lưu mối quan hệ, còn lưu thêm trường nghiệp vụ SoLuong (số lượng sản phẩm nhập). EF Core không tự xử lý trường bổ sung này nếu dùng Many-to-Many thuần.

```
// DoanhNghiepSanPham0214268De2.cs
public int DoanhNghiepId { get; set; } // FK
public DoanhNghiep0214268De2 DoanhNghiep { get; set; }
public int SanPhamId { get; set; } // FK
public SanPham0214268De2 SanPham { get; set; }
[Required]
public int SoLuong { get; set; } // <-- trường nghiệp vụ
```

Composite PK được cấu hình qua Fluent API: HasKey(e => new { e.DoanhNghiepId, e.SanPhamId })

SAI LAM THUONG GAP

Nói là One-to-Many. Không biết tại sao cần entity trung gian (nghĩa Many-to-Many thuần là đủ).

Câu 12

Co ban

CAU HOI

[Table("DoanhNghieps")] trên entity có nghĩa gì? Có cần thiết không?

File: Entities/DoanhNghiep0214268De2.cs

MUC DICH HOI

Kiểm tra hiểu biết về Data Annotations và tên bảng trong DB.

DAP AN MAU

[Table("DoanhNghieps")] là annotation khai báo tương minh tên bảng trong database là "DoanhNghieps".

- Nếu không có: EF Core tự suy ra tên bảng từ tên DbSet property (cùng là "DoanhNghieps" → kết quả giống nhau trong trường hợp này)
- Lợi ích: khai báo tương minh giúp dễ đọc, dễ refactor (đổi tên class mà không đổi tên bảng DB), tránh confusion khi tên bảng khác tên class

```
[Table("DoanhNghieps")]
public class DoanhNghiep0214268De2 : BaseEntity0214268De2
{ ... }
```

SAI LAM THUONG GAP

Nghi annotation này là bắt buộc. Không biết EF Core có quy tắc đặt tên mặc định.

CAU HOI

[Required] va [MaxLength] giúp gì cho EF Core va validation?

File: Entities/ & Dtos/

MUC DICH HOI

Kiểm tra hiểu biết dual-purpose của Data Annotations.

DAP AN MAU

Đối với EF Core (schema DB):

- [Required] → cột NOT NULL trong SQL Server
- [MaxLength(200)] → cột nvarchar(200) thay vì nvarchar(max)

Đối với ASP.NET Core validation (trên DTO):

- [ApiController] tự kiểm tra ModelState trước khi vào action
- Vi phạm → trả 400 Bad Request tự động, không cần if(ModelState.IsValid)

```
// CreateDoanhNghiepDto0214268De2.cs
[Required(ErrorMessage = "Ten doanh nghiep la bat buoc")]
[MaxLength(200, ErrorMessage = "Khong vuot qua 200 ky tu")]
public string TenDoanhNghiep { get; set; }
```

SAI LAM THUONG GAP

Nghĩ annotation chỉ để validate HTTP, không biết ảnh hưởng schema DB. Hoặc ngược lại.

CAU HOI

Navigation property ICollection trong DoanhNghiep để làm gì?

File: Entities/DoanhNghiep0214268De2.cs

MUC DICH HOI

Kiểm tra hiểu biết về navigation properties và cách EF Core sinh SQL JOIN.

DAP AN MAU

Navigation property là thuộc tính cho phép EF Core thực hiện eager/lazy loading để nạp dữ liệu liên quan từ bảng khác.

```
// DoanhNghiep0214268De2.cs
public ICollection<DoanhNghiepSanPham0214268De2> DoanhNghiepSanPhams
{ get; set; } = new List<DoanhNghiepSanPham0214268De2>();
```

Khi cần: .Include(dn => dn.DoanhNghiepSanPhams).ThenInclude(dnsp => dnsp.SanPham) → EF Core sinh SQL JOIN tự động.

Trong dự án này, service dùng LINQ join trực tiếp thay vì Include, nên navigation property ít được gọi trực tiếp nhưng vẫn cần cho EF Core biết cấu hình quan hệ.

SAI LAM THUONG GAP

Nghĩ là trường bình thường, không biết là navigation property. Không biết EF Core dùng nó để sinh JOIN.

PHẦN 3. CONTROLLERS & HTTP

Câu 15

Trung bình

CAU HỎI

[ApiController] trên class Controller mang lại những gì? Nếu bỏ nó thì sao?

File: Controllers/DoanhNghiepController0214268De2.cs

MỤC ĐÍCH HỎI

Kiểm tra hiểu sâu về [ApiController] attribute.

ĐÁP ÁN MÀU

[ApiController] cung cấp các tính năng tự động:

- Auto ModelState validation: nếu [Required] vi phạm → trả 400, không cần viết if (ModelState.IsValid)
- Binding source inference: [FromBody] tự suy ra cho complex type, không cần khai báo tường minh
- Problem Details: định dạng response lỗi theo chuẩn RFC 7807

Nếu bỏ [ApiController]: phải tự kiểm tra ModelState.IsValid ở mọi action, [FromBody] phải khai báo thủ công, response lỗi không theo chuẩn.

```
[Route("api/[controller]")]
[ApiController] // <-- quan trọng
public class DoanhNghiepController0214268De2 : ControllerBase
```

SAI LAM THƯỜNG GẶP

Nghĩ [ApiController] là bắt buộc để dùng routing. Không biết nó tự động xử lý validation.

Câu 16

Trung bình

CAU HỎI

Tại sao controller không gọi DbContext trực tiếp mà gọi Service?

File: Controllers/DoanhNghiepController0214268De2.cs

MỤC ĐÍCH HỎI

Kiểm tra hiểu biết về Separation of Concerns và Single Responsibility Principle.

ĐÁP ÁN MÀU

Separation of Concerns (SoC): mỗi lớp chỉ chịu trách nhiệm 1 việc:

- Controller: xử lý HTTP (nhận request, trả response)
- Service: xử lý business logic (validate nghiệp vụ, CRUD, tính toán)
- DbContext: giao tiếp database

Lợi ích thực tế:

- Dễ viết unit test: mock IDoanhNghiepService mà không cần DB thật
- Tái sử dụng: Service có thể gọi từ nhiều Controller khác nhau
- Dễ thay thế: đổi từ EF Core sang Dapper mà Controller không sửa
- Bảo trì: logic nghiệp vụ tập trung 1 chỗ, không tán mào

SAI LAM THƯỜNG GẶP

Nói chung chung "đề quản lý hơn". Không đưa ra lý do cụ thể có thể kiểm chứng.

CAU HOI

Khi UserFriendlyException duoc nem, controller tra ve HTTP status code nao? Chi ra code.

File: Controllers/DoanhNghiepController0214268De2.cs

MUC DICH HOI

Kiem tra sinh vien doc code va hieu exception handling trong controller.

DAP AN MAU

Controller bat exception trong catch block va tra StatusCode(ex.StatusCode, ...):

```
catch (UserFriendlyException0214268De2 ex)
{
    return StatusCode(ex.StatusCode, new { message = ex.Message });
}
```

- Default: 400 Bad Request (ten trung, MST trung, vi pham nghiep vu)
- Not found: 404 Not Found (khong tim thay doanh nghiep)

Response body: JSON { "message": "..."} }

Exception ngoai UserFriendlyException (NullRef, DbException...) se khong bi bat → tra 500 Internal Server Error mac dinh.

SAI LAM THUONG GAP

Nghi tat ca loi deu tra 400. Khong biet StatusCode duoc lay tu thuoc tinh cua exception.

CAU HỎI

Route [HttpGet("{doanhNghiepId}/san-pham-nhap-nhieu-nhat")] làm gì? Giải thích thuật toán phía sau.

File: Services/Implements/DoanhNghiepService0214268De2.cs

MỤC DICH HỎI

Kiểm tra hiểu biết về routing và logic GetTopSanPham.

DÁP AN MAU

Route này map với URL pattern: GET /api/DoanhNghiep/{id}/san-pham-nhap-nhieu-nhat
Thuật toán phía sau (Service):

1. Kiểm tra DoanhNghiep tồn tại (AnyAsync) → 404 nếu không có
2. Lấy số lượng MAX: MaxAsync((int?)x.SoLuong) — cast nullable tránh lỗi khi rỗng
3. JOIN DoanhNghiepSanPhams + SanPhams lọc SoLuong == maxSoLuong
4. Trả List — có thể nhiều sản phẩm nếu cùng số lượng max

```
// Phan 2 – DoanhNghiepService0214268De2.cs
var maxSoLuong = await _context.DoanhNghiepSanPhams
    .Where(x => x.DoanhNghiepId == doanhNghiepId)
    .MaxAsync(x => (int?)x.SoLuong);
var result = await (from dnsp in _context.DoanhNghiepSanPhams
    join sp in _context.SanPhams
    on dnsp.SanPhamId equals sp.Id
    where dnsp.DoanhNghiepId == doanhNghiepId
    && dnsp.SoLuong == maxSoLuong
    select new SanPhamDto {...}).ToListAsync();
```

SAI LAM THUONG GAP

Nghĩ chỉ trả 1 sản phẩm. Không biết cast (int?) để làm gì (tránh exception khi tập rỗng).

CAU HOI

Tại sao tất cả method trong controller và service đều dùng async/await?

MUC DICH HOI

Kiểm tra hiểu biết về lập trình bất đồng bộ trong ASP.NET Core.

DAP AN MAU

Ly do dung async/await:

- I/O-bound operations (SQL query, doc file) block thread nếu dùng đồng bộ → giảm thông lượng server
- Với async: thread được trả lại thread pool trong lúc chờ DB → server có thể xử lý nhiều request đồng thời hơn với cùng số thread
- ASP.NET Core được thiết kế từ đầu cho async — dùng sync có thể gây deadlock (đặc biệt với .Result/.Wait())

```
// Dung – async
public async Task<IActionResult> Create(...) {
    var result = await _doanhNghiepService.CreateAsync(input);
    return Ok(result);
}

// Sai – có thể gây deadlock trong ASP.NET Core
public IActionResult Create(...) {
    var result = _doanhNghiepService.CreateAsync(input).Result;
    return Ok(result);
}
```

SAI LAM THUONG GAP

Nói "async cho nhanh hơn" mà không giải thích tại sao. Không biết về thread pool và I/O completion.

PHẦN 4. SERVICES & INTERFACES — DEPENDENCY INJECTION

Câu 20

Trung bình

CAU HỎI

Tại sao phải tạo Interface IDoanhNghiepService? Chỉ cần class DoanhNghiepService là đủ?

File: Services/Interfaces/IDoanhNghiepService0214268De2.cs

MỤC ĐÍCH HỎI

Đây là câu hỏi rất hay gap — kiểm tra hiểu biết Dependency Inversion và testability.

DÁP ÁN MẪU

Interface là hợp đồng (contract) giữa Controller và Service. 3 lý do chính:

1. Testability: viết unit test có thể mock interface mà không cần DB thật. Ví dụ: `var mockSvc = new Mock()`
2. Dependency Inversion Principle: Controller phụ thuộc vào abstraction (interface), không phụ thuộc implementation. Để swap implementation không cần sửa Controller.
3. Loose coupling: Controller không biết DoanhNghiepService tồn tại ở đâu, chỉ biết interface — giảm sự phụ thuộc chặt

```
// Nếu không có interface (Bad):  
private readonly DoanhNghiepService0214268De2 _svc; // concrete  
  
// Với interface (Good):  
private readonly IDoanhNghiepService0214268De2 _svc; // abstraction
```

SAI LAM THƯỜNG GẶP

Nói "de de quản lý" hoặc "theo convention". Không biết lợi ích khi test.

CAU HOI

UpdateAsync kiểm tra gì trước khi cập nhật? Điều gì xảy ra nếu Id không tồn tại?

File: Services/Implements/DoanhNghiepService0214268De2.cs

MUC DICH HOI

Kiểm tra sinh viên đọc hiểu UpdateAsync trong Service.

DAP AN MAU

UpdateAsync kiểm tra theo thứ tự:

1. Tìm entity theo Id: `FirstOrDefaultAsync(x => x.Id == input.Id)`
→ Nếu null: `throw UserFriendlyException("Không tìm thấy doanh nghiệp", 404)`
2. Kiểm tra trùng `TenDoanhNghiep` với entity KHAC (loại trừ chính nó):
`AnyAsync(x => x.TenDoanhNghiep == input.TenDoanhNghiep && x.Id != input.Id)`
3. Kiểm tra trùng `MaSoThue` tương tự
4. Gán giá trị mới, `Update()`, `SaveChanges()`, trả DTO

```
// DoanhNghiepService0214268De2.cs
var entity = await _context.DoanhNghieps
    .FirstOrDefaultAsync(x => x.Id == input.Id);
if (entity == null)
    throw new UserFriendlyException0214268De2(
        "Không tìm thấy doanh nghiệp", 404); // 404 not 400
```

SAI LAM THUONG GAP

Quên điều kiện `x.Id != input.Id` khi kiểm tra trùng → báo lỗi sai khi gửi nguyên tên.

CAU HOI

GetListAsync phan trang nhu the nao? Tinh TotalPages o dau?

File: Services/Implements/ & Dtos/PagedResultDto0214268De2.cs

MUC DICH HOI

Kiem tra hieu biet ve phan trang, Skip/Take va cach tinh TotalPages.

DAP AN MAU

Phan trang trong GetListAsync:

```
// DoanhNghiepService0214268De2.cs
var totalCount = await query.CountAsync(); // dem truoc khi phan trang

var items = await query
    .OrderBy(x => x.Id)
    .Skip((input.PageIndex - 1) * input.PageSize) // bo qua trang truoc
    .Take(input.PageSize) // lay dung so luong
    .Select(x => new DoanhNghiepDto {...})
    .ToListAsync();
```

TotalPages duoc tinh trong PagedResultDto0214268De2:

```
// PagedResultDto0214268De2.cs
public int TotalPages =>
    (int)Math.Ceiling((double)totalCount / PageSize);
```

Vi du: TotalCount=25, PageSize=10 → TotalPages = Ceiling(2.5) = 3

SAI LAM THUONG GAP

Tinh TotalPages sai (quen Ceiling hoac de nguyen kieu int chia mat phan du). Dem sau khi phan trang nen sai totalCount.

CAU HOI

Neu muon them DoanhNghiep moi voi ten da ton tai, server tra ve gi?

File: Controllers/ & Services/

MUC DICH HOI

Kiem tra sinh vien hieu flow xu ly loi va HTTP status tra ve.

DAP AN MAU

Flow xu ly:

1. Client POST /api/DoanhNghiep voi TenDoanhNghiep da ton tai
2. Controller.Create() goi Service.CreateAsync()
3. Servicekiem tra: AnyAsync(x => x.TenDoanhNghiep == "...") → true
4. Service throw: new UserFriendlyException("Ten doanh nghiep da ton tai") (StatusCode mac dinh = 400)
5. Controller bat exception:

```
catch (UserFriendlyException0214268De2 ex) {
    return StatusCode(ex.StatusCode, // 400
        new { message = ex.Message }); // "Ten doanh nghiep da ton tai"
}
```

→ Client nhan: HTTP 400 Bad Request, Body: { "message": "Ten doanh nghiep da ton tai" }

SAI LAM THUONG GAP

Noi tra 200 hoac 500. Khong biet StatusCode duoc lay tu exception property.

CAU HOI

Tai sao Servicekiem tra trung ten trong DB thay vi chi dua vao unique index của DB?

File: Services/Implements/DoanhNghiepService0214268De2.cs & DbContexts/

MUC DICH HOI

Cau hoi hay —kiem tra hieu biet ve UX va loi DB exception.

DAP AN MAU

Unique index trong DB la lop bao ve cuoi cung, nhưng neu chi dua vao no:

- SQL Server se throw SqlException (loi DB thuc su), message kho hieu: "Violation of UNIQUE KEY constraint..."
- Controller khong bat SqlException, se tra 500 thay vi 400
- Thong bao loi khong than thien voi user

Kiem tra trong Service truoc → UserFriendlyException(400) → message ro rang, HTTP status dung → UX tot hon.

Ket luan: Unique index la bao ve O DB level (tranh loi race condition khi 2 request dong thoi). Kiem tra trong code la bao ve O application level (UX tot). Can ca 2 lop.

SAI LAM THUONG GAP

Nghi chi can 1 trong 2 la du. Khong hieu race condition va loi tra 500 khi chi dua DB.

PHẦN 5. DTOs & VALIDATION

Câu 25

Co ban

CAU HOI

DTO là gì? Tại sao không truyền Entity trực tiếp từ client?

File: Dtos/DoanhNghiep/CreateDoanhNghiepDto0214268De2.cs

MUC DICH HOI

Câu hỏi cơ bản nhưng rất quan trọng — kiểm tra hiểu biết over-posting và bảo mật.

DAP AN MAU

DTO (Data Transfer Object): đối tượng trung gian chuyển dữ liệu giữa các tầng.

Tại sao không dùng Entity trực tiếp:

1. Over-posting attack: client có thể gửi `IsDeleted=false`, `CreatedAt` tùy ý, hoặc tự đặt `Id`
2. Bảo mật: entity có thể chứa trường nhạy cảm không muốn expose ra ngoài
3. Validation phức tạp: `CreateDto` không cần `Id`, `UpdateDto` bắt buộc có `Id`
4. Phân ly quan tâm: entity là mô hình DB, DTO là mô hình API — 2 khái niệm độc lập
5. Hiệu năng: tránh serialize navigation properties gây circular reference

```
// Nếu dùng Entity trực tiếp (BAD):
POST /api/DoanhNghiep
{"id":999,"tenDoanhNghiep":"Hack","isDeleted":false,"createdAt":"2020-01-01"}
// Client tự đặt Id và CreatedAt → nguy hiểm!

// Dùng DTO (GOOD): chỉ cho phép các trường cần thiết
```

SAI LAM THUONG GAP

Nói "để dễ quản lý code hơn" mà không nêu được over-posting. Không biết circular reference.

CAU HOI

Setter trong DTO tu dong Trim du lieu. Tim dong code nay va giai thich tai sao lam vay.

File: Dtos/DoanhNghiep/CreateDoanhNghiepDto0214268De2.cs

MUC DICH HOI

Kiem tra sinh vien doc code ky va hieu data sanitization.

DAP AN MAU

Dong code trong CreateDoanhNghiepDto0214268De2.cs:

```
private string _tenDoanhNghiep = string.Empty;

public string TenDoanhNghiep
{
    get => _tenDoanhNghiep;
    set => _tenDoanhNghiep = value?.Trim() ?? string.Empty;
    //          ^--- tu dong cat khoang trang
    //          ^--- tranh null
}
```

Tai sao Trim: tranh truong hop " Cong ty ABC " va "Cong ty ABC" duoc coi la 2 ten khac nhau. Dam bao du lieu sach truoc khi luu DB. Khong can lap Trim o nhieu noi trong code. Tai sao ?? string.Empty: tranh NullReferenceException neu client gui null cho truong nay.

SAI LAM THUONG GAP

Khong tim duoc dong code. Nghi Trim o Service hoac Controller.

CAU HOI

CreateDoanhNghiepDto va UpdateDoanhNghiepDto khac nhau the nao? Tai sao can 2 class riêng?

File: Dtos/DoanhNghiep/

MUC DICH HOI

Kiem tra hieu biet DTO design va tai sao Create/Update co truong khac nhau.

DAP AN MAU

Khac nhau: UpdateDoanhNghiepDto co them truong Id bat buoc:

```
// UpdateDoanhNghiepDto0214268De2.cs
[Required(ErrorMessage = "Id la bat buoc")]
[Range(1, int.MaxValue, ErrorMessage = "Id phai lon hon 0")]
public int Id { get; set; } // <-- CHI CO O UPDATE
```

Tai sao can 2 class riêng:

- Create: server tu sinh Id → client khong duoc phep gui Id
- Update: server can biet update ban ghi nao → bat buoc co Id
- Neu dung 1 DTO chung: phai de Id la nullable → validation phuc tap, de sai
- Single Responsibility: moi DTO co validation riêng phu hop voi use-case

SAI LAM THUONG GAP

Nghi co the dung chung 1 DTO, Id la optional. Khong thay su khac biet co Id hay khong.

CAU HOI

FilterDoanhNghiepDto co nhung gi? Keyword duoc su dung de loc theo truong nao?

File: Dtos/DoanhNghiep/FilterDoanhNghiepDto0214268De2.cs & Services/

MUC DICH HOI

Kiem tra sinh vien doc code tim kiem/loc.

DAP AN MAU

FilterDoanhNghiepDto0214268De2 gom:

```
public int PageIndex { get; set; } = 1;    // mac dinh trang 1
public int PageSize { get; set; } = 10;    // mac dinh 10 ban ghi
public string? Keyword { get; set; }       // nullable
// setter tu Trim khoang trang
```

Keyword duoc dung de tim kiem KHONG PHAN BIET HOA THUONG theo 2 truong:

```
// DoanhNghiepService0214268De2.cs
var keyword = input.Keyword.Trim().ToLowerInvariant();
query = query.Where(x =>
    x.TenDoanhNghiep.ToLower().Contains(keyword) ||
    x.MaSoThue.ToLower().Contains(keyword));
```

Dung Contains → tim ket hop (LIKE %keyword%), khong phai exact match.

SAI LAM THUONG GAP

Quen truong MaSoThue. Noi tim ket hop ma thuc ra cung khong biet
Contains = LIKE.

CAU HOI

Mapping giữa DTO và Entity được thực hiện ở đâu? Dự án có dùng AutoMapper không?

File: Services/Implements/DoanhNghiepService0214268De2.cs

MUC DICH HOI

Kiểm tra sinh viên biết vị trí mapping và có thể so sánh với AutoMapper.

DAP AN MAU

Dự án dùng manual mapping thủ công trong DoanhNghiepService0214268De2.cs, KHÔNG dùng AutoMapper:

```
// DTO → Entity (trong CreateAsync)
var entity = new DoanhNghiep0214268De2
{
    TenDoanhNghiep = input.TenDoanhNghiep,
    MaSoThue       = input.MaSoThue,
    DiaChi         = input.DiaChi
    // Id, CreatedAt, IsDeleted: từ dòng từ BaseEntity
};

// Entity → DTO (trong GetListAsync via LINQ .Select)
.Select(x => new DoanhNghiepDto0214268De2
{
    Id           = x.Id,
    TenDoanhNghiep = x.TenDoanhNghiep,
    MaSoThue     = x.MaSoThue,
    DiaChi       = x.DiaChi
})
```

AutoMapper: thư viện tự động map dựa theo tên trường giống nhau, tiết kiệm code nhưng thêm dependency và khó debug khi có lỗi.

SAI LAM THUONG GAP

Nghĩ dự án dùng AutoMapper. Không biết mapping ở Service.

PHAN 6. DbCONTEXT, MIGRATIONS & EF CORE

Câu 30

Co ban

CAU HOI

DbContext là gì? ApplicationDbContext0214268De2 chứa những gì?

File: DbContexts/ApplicationDbContext0214268De2.cs

MUC DICH HOI

Kiểm tra hiểu biết về vai trò trung tâm của DbContext trong EF Core.

DAP AN MAU

DbContext là lớp trung tâm trong EF Core, đóng vai trò:

- Unit of Work: theo dõi thay đổi của các entity (Change Tracking)
- Identity Map: đảm bảo mỗi bản ghi chỉ có 1 instance trong 1 context
- Công vào DB: thực hiện SQL qua SaveChanges()

ApplicationDbContext0214268De2 chứa:

```
public DbSet<DoanhNghiep0214268De2>      DoanhNghieps      { get; set; }
public DbSet<SanPham0214268De2>          SanPhams           { get; set; }
public DbSet<DoanhNghiepSanPham0214268De2> DoanhNghiepSanPhams { get; set; }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Unique indexes, composite PK, cascade delete,
    // global soft-delete filter
}
```

SAI LAM THUONG GAP

Nghĩ DbContext chỉ là lớp để gọi SQL. Không biết Change Tracking.

CAU HOI

Migrations la gi? Du an co bao nhieu migration va moi migration lam gi?

File: Migrations/

MUC DICH HOI

Kiem tra hieu biet ve Code-First va lich su schema DB.

DAP AN MAU

Migration la file C# ghi lai thay doi schema DB theo tung moc thoi gian (Code-First approach).

Du an co 2 migration:

1. InitialCreateClean0214268De2 (20260528041720): tao 3 bang (DoanhNghieps, SanPhams, DoanhNghiepSanPhams) + unique indexes + FK
2. SyncPendingChanges0214268De2 (20260528052702): xoa cot Id khoi DoanhNghiepSanPhams, doi PK thanh composite (DoanhNghiepId, SanPhamId)

```
// Tao migration moi:
dotnet ef migrations add TenMigration

// Ap dung len DB:
dotnet ef database update

// Lich su migration duoc luu trong bang:
__EFMigrationsHistory
```

SAI LAM THUONG GAP

Nghi migration tu dong chay khi start app. Khong biet co 2 migration va moi cai lam gi.

CAU HỎI

OnModelCreating cấu hình những gì? Chỉ ra cấu hình quan trọng nhất.

File: *DbContexts/ApplicationDbContext0214268De2.cs*

MỤC ĐÍCH HỎI

Kiểm tra hiểu biết về Fluent API và cấu hình EF Core.

DÁP AN MẪU

OnModelCreating cấu hình 4 việc:

1. Unique indexes cho DoanhNghiep (TenDoanhNghiep, MaSoThue) và SanPham (TenSanPham, MaSanPham)
2. Composite Primary Key cho DoanhNghiepSanPham: HasKey(e => new { e.DoanhNghiepId, e.SanPhamId })
3. Cascade delete: khi xóa DoanhNghiep → tự xóa DoanhNghiepSanPham liên quan
4. Global query filter (quan trọng nhất): tự động thêm WHERE IsDeleted=false vào mọi query

```
// Global soft-delete filter:
foreach (var entityType in modelBuilder.Model.GetEntityTypes())
{
    if (typeof(BaseEntity0214268De2)
        .IsAssignableFrom(entityType.ClrType))
    {
        // Tao expression: e => e.IsDeleted == false
        modelBuilder.Entity(entityType.ClrType)
            .HasQueryFilter(filter);
    }
}
```

SAI LAM THƯỜNG GẶP

Chỉ biết một phần. Không biết global query filter hoặc cascade delete.

CAU HOI

Khi thêm trường Email vào DoanhNghiep, phải làm gì để cập nhật DB?

File: Migrations/

MUC DICH HOI

Kiểm tra hiểu biết về quy trình Code-First migration.

DAP AN MAU

Quy trình cập nhật DB (Code-First):

```
Buoc 1: Thêm trường vào entity
// DoanhNghiep0214268De2.cs
[MaxLength(100)]
public string Email { get; set; } = string.Empty;

Buoc 2: Tạo migration mới
dotnet ef migrations add AddEmailToDoanhNghiep
// → EF Core sẽ sinh DbContext vs ModelSnapshot
// → Sinh file migration:
// Up(): AddColumn("Email", "DoanhNghieps", ...)
// Down(): DropColumn("Email", "DoanhNghieps")

Buoc 3: Áp dụng lên DB
dotnet ef database update
// → Chạy Up() của migration mới
// → Ghi vào __EFMigrationsHistory

Buoc 4: ModelSnapshot tự cập nhật
```

SAI LAM THUONG GAP

Nghĩ chỉ cần sửa entity là DB tự cập nhật. Không biết phải chạy 2 lệnh ef.

Câu 34

Nâng cao

CAU HOI

Neu không dùng Migration, có cách nào khác để tạo/cập nhật schema DB không?

MUC DICH HOI

Câu hỏi phân biệt — kiểm tra hiểu biết các phương án thay thế.

DAP AN MAU

Có 3 phương án thay thế Migration:

1. Database.EnsureCreated(): EF Core tự tạo bảng dựa trên model hiện tại, KHÔNG ghi lịch sử migration. Chỉ dùng cho prototype/dev, không dùng production.
2. Database.Migrate(): áp dụng tất cả migration chưa chạy khi app khởi động (thay vì dùng CLI). Có thể dùng trong startup nhưng rủi ro.
3. Viết SQL tay: tạo script .sql thực thi thủ công — mất lịch sử, khó quản lý

Ưu điểm của Migration: có lịch sử rõ ràng, có thể rollback (Down()), kiểm soát được thay đổi schema theo từng phiên bản.

SAI LAM THUONG GAP

Nghĩ Migration là cách duy nhất. Không biết EnsureCreated hay Migrate().

CAU HOI

[ForeignKey("DoanhNghiepId")] tren navigation property co nghĩa gì?

File: Entities/DoanhNghiepSanPham0214268De2.cs

MUC DICH HOI

Kiểm tra hiểu biết về Data Annotations cho FK trong EF Core.

DAP AN MAU

[ForeignKey("DoanhNghiepId")] khai báo tuồng minh rằng DoanhNghiepId là foreign key của navigation property DoanhNghiep.

```
// DoanhNghiepSanPham0214268De2.cs
public int DoanhNghiepId { get; set; } // FK column
[ForeignKey("DoanhNghiepId")] // <-- annotation
public DoanhNghiep0214268De2 DoanhNghiep { get; set; } = null!;
```

Neu không có annotation, EF Core sẽ tự đặt tên ClassName + "Id" (convention) — là "DoanhNghiepId" → cũng kết quả. Annotation giúp tuồng minh hơn, tránh nhập nhò khi tên không theo convention.

null! (null-forgiving): báo compiler rằng property này sẽ được gán giá trị bởi EF Core khi load từ DB, tránh cảnh báo nullable.

SAI LAM THUONG GAP

Không biết ForeignKey annotation. Không hiểu null! có nghĩa gì.

PHẦN 7. EXCEPTIONS, UTILS & CONSTANTS

Câu 36

Trung bình

CAU HỎI

UserFriendlyException kế thừa từ gì? Tại sao cần tạo exception riêng?

File: Exceptions/UserFriendlyException0214268De2.cs

MỤC ĐÍCH HỎI

Kiểm tra hiểu biết về custom exception và tại sao phân biệt lỗi nghiệp vụ vs lỗi hệ thống.

DÁP AN MÀU

UserFriendlyException0214268De2 kế thừa từ System.Exception:

```
public class UserFriendlyException0214268De2 : Exception
{
    public int StatusCode { get; set; }

    public UserFriendlyException0214268De2(
        string message,
        int statusCode = 400) // mặc định 400
        : base(message)
    {
        StatusCode = statusCode;
    }
}
```

Tại sao cần exception riêng:

- Phân biệt lỗi nghiệp vụ (UserFriendly — biết trước, trả 400/404) với lỗi hệ thống (NullRef, DbException — không lường trước, trả 500)
- Controller chỉ bắt UserFriendlyException → lỗi hệ thống sẽ trả 500 phù hợp
- Mang theo StatusCode → controller biết trả HTTP status nào mà không cần if-else

SAI LAM THƯỜNG GẶP

Nghĩ tất cả exception đều giống nhau. Không hiểu tại sao chỉ bắt UserFriendlyException.

CAU HỎI

AppConstants0214268De2 chưa gì? Các DTO có sử dụng constants này không? Nếu không tại sao?

File: *Constants/AppConstants0214268De2.cs* & *Dtos/*

MỤC ĐÍCH HỎI

Câu hỏi phát hiện sự không nhất quán — kiểm tra sinh viên đọc code rất kỹ.

DÁP AN MAU

AppConstants0214268De2 khai báo:

```
public const int DefaultPageSize      = 10;
public const int MaxPageSize          = 100;
public const int MaxTenDoanhNghiepLength = 200;
public const int MaxMaSoThueLength    = 50; // ...
```

Điểm quan trọng: Các DTO **KHÔNG** sử dụng constants này! Thay vào đó, DTO hard-code trực tiếp:

```
// FilterDoanhNghiepDto0214268De2.cs
public int PageSize { get; set; } = 10; // hard-code, KHÔNG dùng AppConstants.DefaultPageSi
ze

// CreateDoanhNghiepDto0214268De2.cs
[MaxLength(200)] // hard-code, KHÔNG dùng AppConstants.MaxTenDoanhNghiepLength
```

Đây là điểm có thể bị hỏi phản biện: **AppConstants** được tạo ra nhưng không được sử dụng → nếu thay đổi giá trị, phải sửa cả 2 nơi → dễ bug.

Cách sửa tốt: thay `MaxLength(200)` bằng `MaxLength(AppConstants0214268De2.MaxTenDoanhNghiepLength)`

SAI LAM THUONG GAP

Nói "có sử dụng" mà thực ra không. Đây là câu bẫy rất hay.

CAU HOI

StringUtils0214268De2 chưa gì? Được gọi ở đâu trong dự án?

File: Utils/StringUtils0214268De2.cs

MUC DICH HOI

Kiểm tra sinh viên đọc hiểu Utils và phát hiện nó ít được dùng.

DAP AN MAU

StringUtils0214268De2 có 2 method static:

```
public static string TrimOrEmpty(string? input)
    => input?.Trim() ?? string.Empty;

public static bool IsNullOrEmpty(string? input)
    => string.IsNullOrEmpty(input);
```

Thực tế trong dự án: StringUtils IT DUOC GOI TRUC TIEP vì logic Trim đã được tích hợp trực tiếp vào setter của mỗi DTO:

```
// CreateDoanhNghiepDto0214268De2.cs
set => _tenDoanhNghiep = value?.Trim() ?? string.Empty;
// Tương đương StringUtils.TrimOrEmpty() nhưng viết trực tiếp
```

Đây là điểm có thể phát hiện: khai báo utility class nhưng code thực tế không gọi → code smell (dead code hoặc thiếu sử dụng nhất quán).

SAI LAM THUONG GAP

Nghi StringUtils được gọi nhiều. Không biết logic Trim đã ở setter DTO.

PHẦN 8. CÂU HỎI "BAY" — GIẢNG VIÊN DÙNG ĐỂ KIỂM TRA HIỂU BIẾT THẬT

Đây là những câu giảng viên thường hỏi để xem sinh viên có thực sự hiểu code hay chỉ học vẹt.

Câu 39

Bay

CÂU HỎI

BAY: Nếu bỏ interface `IDoanhNghiepService`, chỉ để class `DoanhNghiepService`, code có chạy không? Khác gì?

MỤC ĐÍCH HỎI

Giảng viên muốn xem sinh viên có hiểu interface là optional hay bắt buộc.

ĐÁP ÁN MAU

Code vẫn chạy bình thường nếu bỏ interface — vì DI container có thể inject concrete class trực tiếp.

Khác biệt thực tế:

- Mặt khả năng mock khi unit test (không thể `Mock<>` concrete class có virtual methods giới hạn)
- Controller bị ràng buộc với implementation cụ thể → vi phạm DIP
- Muốn thay `DoanhNghiepService` bằng `CachedDoanhNghiepService` phải sửa toàn bộ nơi inject

```
// Nếu bỏ interface:
private readonly DoanhNghiepService0214268De2 _svc; // concrete
// → Kho test, kho mô phỏng

// Giữ interface:
private readonly IDoanhNghiepService0214268De2 _svc; // abstraction
// → Để test (mock), để thay thế implementation
```

SAI LAM THƯỜNG GẶP

Nói "không chạy được" — SAI, code vẫn chạy. Hoặc nói "giống nhau" — SAI, mất testability.

CAU HOI

BAY: AppConstants được tạo ra nhưng các DTO không dùng nó. Đây có phải bug không?

File: Constants/AppConstants0214268De2.cs

MUC DICH HOI

Giang vien muon thay sinh vien co doc code that ky va phat hien su khong nhac quan.

DAP AN MAU

Đây KHÔNG phải bug — ứng dụng vẫn chạy đúng. Nhưng đây là code smell (mùi code):

- Dead code / Unused code: AppConstants được khai báo nhưng không được dùng trong DTO
- Magic numbers: DTO hard-code 10, 100, 200, 50 trực tiếp mà không tham chiếu constants → nếu sau này muốn thay đổi MaxLength từ 200 thành 300, phải sửa cả entity lẫn DTO
- Single Source of Truth bị vi phạm: giá trị 200 xuất hiện ở nhiều nơi (Constants + DTO + Entity)

Cách sửa tốt nhất:

```
// DTO:
[MaxLength(AppConstants0214268De2.MaxTenDoanhNghiepLength)]

// FilterDto:
public int PageSize { get; set; }
    = AppConstants0214268De2.DefaultPageSize;
```

SAI LAM THUONG GAP

Nói "không sao, vẫn chạy được" mà không phát hiện vấn đề thiết kế. Đây là câu phản biện mạnh.

CAU HOI

BAY: Global query filter cho soft delete có vấn đề gì không? Có trường hợp nào bị lọc oan không?

File: DbContexts/ApplicationDbContext0214268De2.cs

MUC DICH HOI

Câu hỏi nâng cao — kiểm tra hiểu biết sâu về global query filter và giới hạn của nó.

DAP AN MAU

Vấn đề tiềm ẩn của global query filter:

1. JOIN query bị ảnh hưởng: khi join giữa DoanhNghiep (có filter) và DoanhNghiepSanPham (có filter), cả 2 bảng đều tự động thêm WHERE IsDeleted=false → có thể bỏ sót dữ liệu mong muốn
2. Admin cần xem bản ghi đã xóa: phải gọi IgnoreQueryFilters() tự động → dễ quên
3. MaxAsync() trên dữ liệu đã xóa: nếu muốn tính max toàn bộ (kể cả deleted), phải biết bypass filter
4. Cascade delete + soft delete: khi soft delete DoanhNghiep, DoanhNghiepSanPham liên quan bị cascade delete VAT LY (vi dụ dùng OnDelete(Cascade) thực sự) → bất nhất với soft delete

```
// Để xem dữ liệu đã soft-delete:
var all = await _context.DoanhNghieps
    .IgnoreQueryFilters() // bypass filter
    .ToListAsync();
```

SAI LAM THUONG GAP

Nghi global filter là hoàn hảo, không có nhược điểm. Đây là câu phản biện mạnh.

CAU HOI

BAY: Neu 2 request dong thoi POST voi cung TenDoanhNghiep, co the xay ra race condition khong?

File: Services/ & DbContexts/

MUC DICH HOI

Kiem tra hieu biet sau nhat ve concurrent access va database constraints.

DAP AN MAU

CO the xay ra race condition:

Kich ban: Request A va B cung POST ten "Cong ty XYZ" cung luc

```
Request A: AnyAsync("Cong ty XYZ") → false (ten chua ton tai)
Request B: AnyAsync("Cong ty XYZ") → false (ten chua ton tai)
Request A: INSERT "Cong ty XYZ" → thanh cong
Request B: INSERT "Cong ty XYZ" → ???
```

- Neu chi co check trong Service: ca 2 deu vuot qua AnyAsync, ca 2 deu INSERT → DU LIEU TRUNG
 - Lop bao ve cuoi: Unique Index tren TenDoanhNghiep trong DB → Request B se bi Lo SqlException (Unique Key violation)
 - Controller KHONG bat SqlException → tra 500 Internal Server Error (trai voi mong doi)
- Cach xu ly dung: Them try-catch SqlException hoac dung database transaction voi isolation level cao hon.

SAI LAM THUONG GAP

Nghi AnyAsync() bao ve hoan toan. Khong biet race condition. Day la cau cuc kho.

CAU HOI

BAY: Tai sao UpdatedAt trong BaseEntity la DateTime? (nullable) trong khi CreatedAt la DateTime?

File: Entities/BaseEntity0214268De2.cs

MUC DICH HOI

Kiem tra sinh vien hieu thiet ke entity va y nghia cua nullable.

DAP AN MAU

- CreatedAt = DateTime (khong nullable): MOI ban ghi deu co thoi diem tao → KHONG bao gio null. Duoc gan mac dinh = DateTime.UtcNow ngay khi khai tao entity.
- UpdatedAt = DateTime? (nullable): ban ghi MOI TAO chua bao gio duoc update → UpdatedAt = null la hop le. Chi co gia tri khi da tung duoc cap nhat.

```
// BaseEntity0214268De2.cs
public DateTime CreatedAt { get; set; } = DateTime.UtcNow; // KHONG null
public DateTime? UpdatedAt { get; set; } // co the null
```

Luu y: Trong UpdateAsync cua du an nay, UpdatedAt KHONG duoc gan gia tri hien tai khi update → day la thieu sot thiet ke co the bi hoi phan bien.

SAI LAM THUONG GAP

Nghi 2 truong la giong nhau. Day la cau phat hien thieu sot trong UpdateAsync.

CAU HOI

BAY: Du an KHONG co middleware xu ly exception toan cuc. Hau qua la gi?

File: Program.cs (thieu middleware)

MUC DICH HOI

Phat hien thieu sot quan trong trong kien truc — kiểm tra sinh viên đọc code rat ky.

DAP AN MAU

Du an THIEU global exception handling middleware. Hau qua:

- Neu Service throw `SqlException` hoac `NullReferenceException` (khong phai `UserFriendlyException`) → Controller KHONG bat → ASP.NET Core tra 500 Internal Server Error voi stack trace (neu dev mode) hoac rong (production)
- Stack trace bi lo ra response → bao mat kem (tiet lo cau truc code)
- Khong co logging tap trung → kho debug production

Cach sua: Them Global Exception Middleware:

```
// Middleware:
app.UseExceptionHandler(appError =>
{
    appError.Run(async context =>
    {
        context.Response.StatusCode = 500;
        await context.Response.WriteAsJsonAsync(
            new { message = "Loi he thong" });
        // log loi o day
    });
});
```

Hoac them `ILogger` vao Service de log truoc khi nem exception.

SAI LAM THUONG GAP

Nghi cach xu ly loi hien tai la du. Khong biet du an thieu middleware quan trong.

PHẦN 9. KỊCH BẢN VĂN ĐÁP THỰC TẾ — 30 CÂU LIÊN TIẾP

Mô phỏng buổi văn đáp 30 phút. GV hỏi tiếp dựa trên câu trả lời trước.

GV = Giảng viên | SV = Sinh viên (bạn)

GV: (1/30) Giới thiệu ngắn gọn về dự án của bạn.

SV: Đây là dự án ASP.NET Core Web API trên .NET 10, quản lý Doanh Nghiệp và Sản Phẩm. Dự án có kiến trúc 3 tầng: Controllers nhận request, Services xử lý nghiệp vụ, DbContext giao tiếp SQL Server qua EF Core. Dự án dùng soft delete, phân trang, tìm kiếm theo keyword.

GV: (2/30) Dự án dùng gì để kết nối database?

SV: Entity Framework Core với provider Microsoft.EntityFrameworkCore.SqlServer. Connection string được cấu hình trong appsettings.json, bảng ApplicationDbContext0214268De2 kế thừa DbContext.

GV: (3/30) Program.cs có gì đặc biệt? Giải thích nhanh.

SV: Program.cs đăng ký 3 thứ: (1) DbContext với SQL Server, (2) Scoped DI cho IDoanhNghiepService, (3) Swagger. Pipeline có middleware UseSwagger, UseAuthorization, và redirect "/" sang "/swagger".

GV: (4/30) Tại sao AddScoped cho Service mà không phải AddSingleton?

SV: DbContext là Scoped (1 instance / request). Service inject DbContext nên Service cũng phải Scoped. Nếu Service là Singleton nhưng inject Scoped DbContext, DI container sẽ báo lỗi "Captive Dependency".

Lưu ý: Captive Dependency — Singleton chưa Scoped là lỗi bắt buộc phải biết

GV: (5/30) Nếu tôi gọi POST /api/DoanhNghiep với body rỗng {}, server trả gì?

SV: [ApiController] tự động kiểm tra ModelState. [Required] vi phạm → trả 400 Bad Request với danh sách lỗi chưa ErrorMessage tung trường. Không cần viết code xử lý thủ công.

GV: (6/30) DTO có gì đặc biệt không phải là POCO thông thường?

SV: Setter của từng trường tự động Trim whitespace: set => _tenDoanhNghiep = value?.Trim() ?? string.Empty. Đây là data sanitization ngay khi binding, trước cả validation.

GV: (7/30) Nếu client gửi TenDoanhNghiep là " " (toàn khoảng trắng), [Required] có bắt không?

SV: Có vấn đề: Sau khi Trim, TenDoanhNghiep = string.Empty (chuỗi rỗng). [Required] chỉ kiểm tra khác null, không kiểm tra rỗng. Tuy nhiên [MinLength(1)] sẽ bắt trường hợp này và trả 400.

Đây là câu hay — [Required] và [MinLength(1)] làm việc cùng nhau

GV: (8/30) CreateAsync kiểm tra gì trước khi lưu?

SV: Kiểm tra 2 điều: (1) TenDoanhNghiep có bị trùng với bản ghi khác không bằng AnyAsync. (2) MãSoThue có bị trùng không. Nếu trùng bất kỳ trường hợp nào → throw UserFriendlyException(400). Nếu OK thì mới tạo entity và SaveChanges.

GV: (9/30) Sau khi SaveChanges, entity.Id có giá trị không?

SV: Có. Sau SaveChangesAsync(), EF Core cập nhật lại entity object với Id được SQL Server trả về (IDENTITY column). Đây là lý do tại sao return new DoanhNghiepDto { Id = entity.Id } cho giá trị Id chính xác.

GV: (10/30) Soft delete được implement như thế nào trong DeleteAsync?

SV: Thay vì gọi `Remove()` hoặc `SQL DELETE`, `DeleteAsync` đặt `entity.IsDeleted = true` rồi gọi `Update()` và `SaveChanges()`. Bạn ghi vào con trong DB nhưng global query filter tự động loại khỏi mọi query tiếp theo.

GV: (11/30) Global query filter hoạt động như thế nào?

SV: Trong `OnModelCreating`, `DbContext` dùng reflection duyệt tất cả entity kế thừa `BaseEntity`. Với mọi entity đó, đặt `HasQueryFilter` với expression `e => e.IsDeleted == false`. EF Core tự động thêm điều kiện này vào `SQL WHERE` clause.

Hỏi thêm: "Có thể bypass filter không?" → trả lời: `IgnoreQueryFilters()`

GV: (12/30) GetListAsync phân trang như thế nào? Tính TotalPages ở đâu?

SV: Phân trang bằng `Skip((PageIndex-1)*PageSize).Take(PageSize)`. `TotalPages` được tính trong `PagedResultDto` qua property computed: `(int)Math.Ceiling((double)TotalCount / PageSize)`.

GV: (13/30) Khi tìm kiếm với Keyword, có phân biệt hoa thường không?

SV: Không phân biệt. Code dùng `ToLower().Contains(keyword.ToLowerInvariant())` — convert cả 2 phía về lowercase trước khi so sánh. Đây là case-insensitive search.

GV: (14/30) Bảng DoanhNghiepSanPham có Primary Key là gì?

SV: Composite Primary Key: (`DoanhNghiepId`, `SanPhamId`). Được cấu hình qua Fluent API: `HasKey(e => new { e.DoanhNghiepId, e.SanPhamId })`. Migration thứ 2 đã xóa ID column đơn và thêm composite PK này.

GV: (15/30) Tại sao migration thứ 2 xuất hiện? Không làm gì?

SV: Migration `InitialCreate` tạo `DoanhNghiepSanPhams` với PK là cột ID đơn (vì EF Core tự thêm ID khi kế thừa `BaseEntity`). Nhưng thực tế bảng này cần composite PK. Migration thứ 2 sửa lại: drop PK cũ, xóa cột ID, thêm composite PK mới.

Đây là dấu hiệu sinh viên thực sự debug và có hiểu biết thực tế

GV: (16/30) Cascade delete có nghĩa gì trong dự án này?

SV: `OnDelete(Cascade)`: khi xóa `DoanhNghiep`, EF Core tự động xóa tất cả `DoanhNghiepSanPham` có `DoanhNghiepId` tương ứng. Tuy nhiên dự án dùng soft delete (`IsDeleted=true`), nên cascade delete vật lý tạo ra bất nhất.

Đây là điểm có thể bị phân biệt — soft delete + cascade delete thật sự bất nhất

GV: (17/30) GetTopSanPhamByDoanhNghiepIdAsync có thể trả về nhiều sản phẩm không?

SV: Có. Thuật toán: (1) Lấy MAX `SoLuong`, (2) Lấy TẤT CẢ sản phẩm có `SoLuong == MAX`. Nếu 3 sản phẩm cùng nhập 100 chiếc, trả về cả 3. Kết quả là List không giới hạn số lượng.

GV: (18/30) Tại sao MaxAsync dùng (int?)x.SoLuong thay vì x.SoLuong?

SV: Cast sang nullable int? để tránh exception khi doanh nghiệp chưa có sản phẩm nào (tập rỗng). `MaxAsync` trên tập rỗng với kiểu non-nullable sẽ throw `InvalidOperationException`. Nullable cho phép trả về null thay vì exception, sau đó kiểm tra null để return list rỗng.

Đây là câu kỹ thuật hay — sinh viên mà biết cái này là hiểu sâu

GV: (19/30) UserFriendlyException khác gì Exception thông thường?

SV: `UserFriendlyException` thêm thuộc tính `StatusCode` (mặc định 400). Controller chỉ bắt `UserFriendlyException` để trả lời rõ ràng cho client. Exception khác (`SqlException`, `NullRef`) không bị bắt → trả 500. Phân tách lỗi "biết trước" và "không lường trước".

GV: (20/30) Unique Index tren TenDoanhNghiep va MaSoThue trong DB co tac dung gi khi service da kiem tra roi?

SV: Unique Index la lop bao ve cuoi cung o DB level, bao ve khi race condition khi 2 request dong thoi vuot qua check cua Service. Tuy nhien du an chua xu ly SqlException nay → co the tra 500 thay vi 400 trong tinh huong dong thoi.

Sinh vien biet dieu nay se gay an tuong manh voi giang vien

GV: (21/30) Noi den 1 diem yeu hoac han che cua du an ma ban thay.

SV: Du an thieu global exception handling middleware — loi khong phai UserFriendlyException se tra 500 voi stack trace trong dev mode. Ngoai ra UpdatedAt khong duoc cap nhat trong UpdateAsync. AppConstants duoc khai bao nhung khong duoc tham chieu trong DTO.

Sinh vien tu phat hien diem yeu = diem cong cuc lon

GV: (22/30) Lam the nao de them logging vao du an nay?

SV: Inject ILogger qua constructor cua Service. Them them app.UseExceptionHandler() middleware trong Program.cs de log exception toan cuc. Hien tai du an chua co ILogger o bat ky lop nao.

GV: (23/30) Neu khach hang muon them chuc nang xuat danh sach ra Excel, ban them code o dau?

SV: Them method ExportExcelAsync trong IDoanhNghiepService va DoanhNghiepService. Them action ExportExcel trong Controller. Them thu vien nhu ClosedXML hoac EPPlus. Khong sua controller khac, khong sua entity hay migration.

GV: (24/30) Tai sao ControllerBase thay vi Controller?

SV: ControllerBase la lop co so cho API Controller — chi co cac phuong thuc HTTP (Ok, NotFound, BadRequest...). Controller ke thua ControllerBase va them View support (cho MVC voi Razor). API khong can render View → dung ControllerBase la dung, gon hon.

GV: (25/30) [Route("api/[controller]")] → [controller] se duoc thay the bang gi?

SV: [controller] la token tu dong — ASP.NET Core lay ten class, bo chu "Controller" o cuoi. DoanhNghiepController0214268De2 → bo "Controller" va suffix "0214268De2"... thuc ra lay phan truoc "Controller" → "DoanhNghiep". URL se la /api/DoanhNghiep.

GV: (26/30) Neu can them entity SanPham CRUD, ban them nhung gi?

SV: Can them: (1) CreateSanPhamDto, UpdateSanPhamDto, FilterSanPhamDto trong Dtos/SanPham/, (2) ISanPhamService trong Interfaces/, (3) SanPhamService trong Implements/, (4) SanPhamController trong Controllers/, (5) Dang ky DI trong Program.cs. Khong can sua migration neu khong them truong moi vao SanPham.

GV: (27/30) ICollection duoc khoi tao bang new List<> trong entity. Tai sao?

SV: Tranh NullReferenceException khi truy cap navigation property ma chua load tu DB. Neu de null, code nhu doanhNghiep.DoanhNghiepSanPhams.Count() se bao NullRef. Khoi tao rong list dam bao luon co the iterate an toan.

GV: (28/30) Ban cung co the mo ta SQL duoc EF Core sinh ra khi goi GetListAsync voi Keyword khong?

SV: SQL tuong duong: SELECT TOP(@p2) Id, TenDoanhNghiep, MaSoThue, DiaChi FROM DoanhNghieps WHERE IsDeleted = 0 AND (LOWER(TenDoanhNghiep) LIKE @p0 OR LOWER(MaSoThue) LIKE @p1) ORDER BY Id OFFSET @p3 ROWS. IsDeleted=0 den tu global filter. LIKE den tu Contains().

GV: (29/30) Neu giang vien yeu cau deploy len Azure, ban can thay doi gi?

SV: Chinh sua connection string trong appsettings.json hoac Environment Variables tren Azure (Server=..., ten DB tren Azure SQL). Co the can them TrustServerCertificate=False. Cai dat dotnet publish, config App Service tren Azure Portal. Migration co the chay qua database update hoac startup code.

GV: (30/30) Cuoi cung, neu co them 1 ngay de cai thien du an, ban lam gi dau tien?

SV: Thu nhat them global exception handling middleware trong Program.cs (UseExceptionHandler) de tranh lo stack trace va tra 500 khong ro rang. Thu hai cap nhat UpdatedAt trong UpdateAsync. Thu ba tham chieu AppConstants trong DTO de dam bao Single Source of Truth.

Sinh vien nhan ra va phat bieu diem yeu mot cach chu dong = diem cong rat lon

TONG KET. NHUNG DIEM QUAN TRONG NHAT CAN NHO

#	Diem can nho	File lien quan
1	Soft delete: IsDeleted=true, global filter tu dong loai	BaseEntity + DbContext
2	DI: AddScoped vi DbContext la Scoped (tranh Captive Dep.)	Program.cs
3	Interface: testability + DIP + loose coupling	Services/Interfaces/
4	DTO setter tu Trim: value?.Trim() ?? string.Empty	Dtos/DoanhNghiep/
5	UserFriendlyException mang StatusCode (400/404)	Exceptions/
6	[ApiController] tu validate ModelState → tra 400 tu dong	Controllers/
7	Composite PK: HasKey(e => new { e.DoanhNghiepId, e.SanPhamId })	DbContexts/ + Migrations/
8	Race condition: Service check + Unique Index = 2 lop bao ve	Services/ + DbContexts/
9	AppConstants khai bao nhung DTO khong dung → thieu nhat quan	Constants/ + Dtos/
10	UpdatedAt khong duoc cap nhat trong UpdateAsync → thieu sot	Services/Implements/
11	Thieu global exception middleware → loi 500 bi lo stack trace	Program.cs (thieu)
12	MaxAsync(x => (int?)x.SoLuong): cast nullable tranh exception tap rong	Services/Implements/

Chuc ban van dap thanh cong!